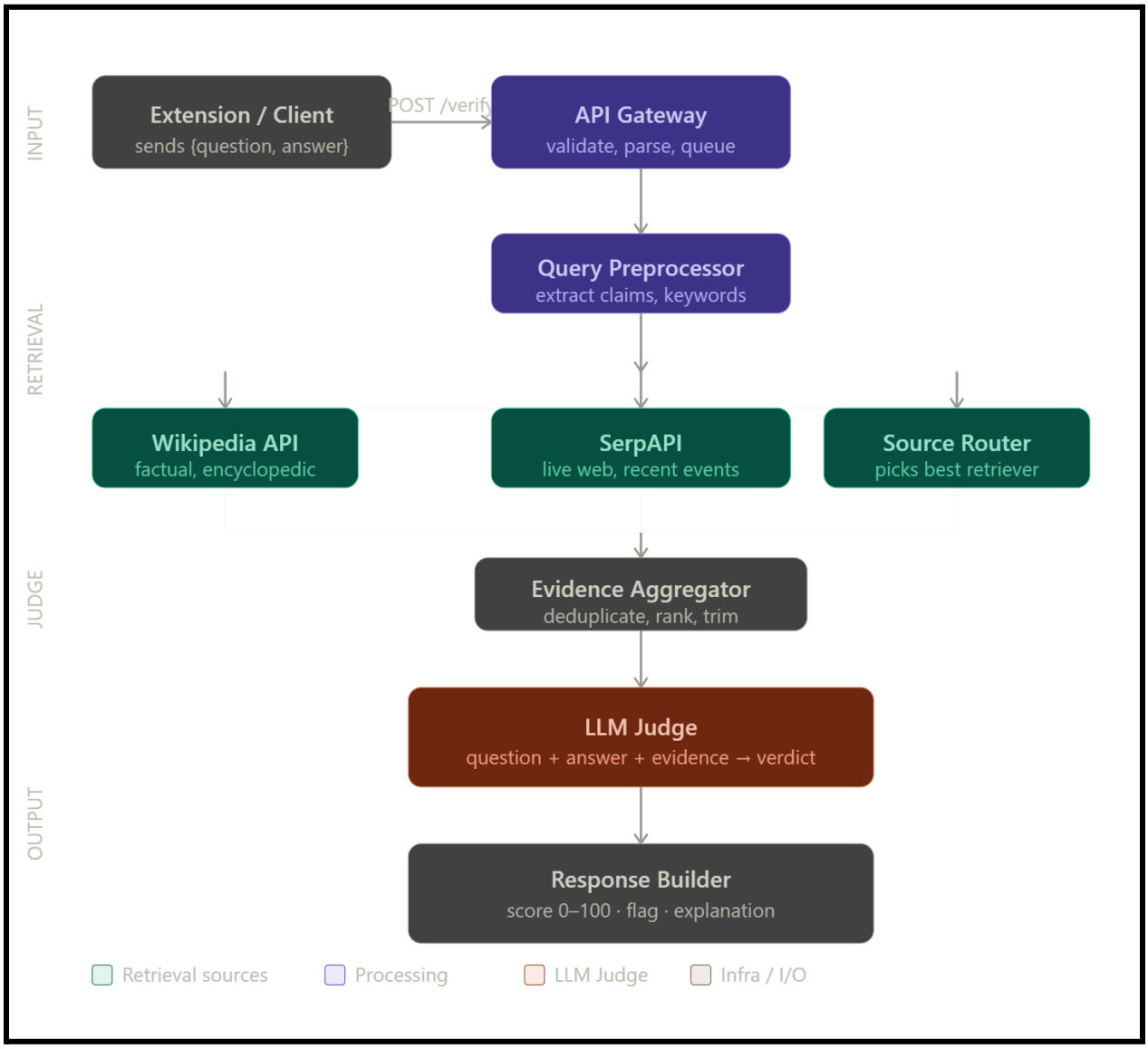
This is a solid, well-reasoned approach. The evidence-grounded LLM judge is significantly better than a pure GPT-as-a-Judge setup because it reduces the judge's own hallucination risk. Let me validate it and then lay out a clean implementation plan.

What's strong about your approach:

- External evidence retrieval grounds the judgment in real-world facts
- Separating retrieval from judgment keeps concerns cleanly separated
- Using a second LLM as judge (not the same one) avoids self-validation bias

One gap to address: Wikipedia and SerpAPI have different reliability profiles. Wikipedia is great for factual/encyclopedic claims but misses recent events and niche topics. SerpAPI gives you live web results but quality varies. You'll want a retrieval strategy that picks the right source based on the query type.

Here's a proper implementation plan for the backend:Here's the full architecture diagram first, then I'll lay out each layer of the implementation plan.



Now here's the full implementation plan, layer by layer.

Implementation Plan

Layer 1 — API Gateway (`POST /verify`)

This is the entry point. It receives a JSON payload from the client (your extension later, Postman for now) and does three things: validates the input, extracts it into a clean internal format, and hands it off to the pipeline.

Payload contract:

```
{ "question": "...", "answer": "..." }
```

Tech: FastAPI (Python) is ideal here — fast to build, async-native, auto-generates docs. One endpoint, one Pydantic model for validation. Return 422 on bad input automatically.

Layer 2 — Query Preprocessor

Before you call any external API, you need to figure out *what claims* are actually being made in the answer. Don't just dump the full answer into a Wikipedia search — it'll return garbage.

What it does:
- Extracts the key factual claims from the answer (1–3 claims max per request)
- Identifies the query type: *encyclopedic* (person, place, concept), *recent event*, *numeric/statistical*, or *opinion/subjective*
- Routes to the right retriever(s) based on type

Implementation choice: Use a small LLM call here (or regex + heuristics to start). Prompt: *"Extract the 2–3 most verifiable factual claims from this answer as short search queries."* This is cheap and fast.

Layer 3 — Retrieval Engine

This is the most critical part. The three components:

Wikipedia API — use `wikipedia-api` Python library. Best for: named entities, historical facts, scientific concepts, biographical info. Call it with the extracted claim as a search query. Grab the top 2 paragraphs of the most relevant article.

SerpAPI — live Google search results. Best for: recent events (< 1 year), statistics, current affairs. Grab the top 3 snippets. This costs money per call, so gate it: only call SerpAPI if Wikipedia returns nothing useful OR the query type is "recent."

Source Router — a simple decision function (not another LLM call) that decides which retriever(s) to invoke based on the claim type detected in Layer 2. This keeps costs down.

Evidence Aggregator — takes results from all retrievers, deduplicates, removes boilerplate, and trims to ~800 tokens of context so you don't blow the LLM judge's context window.

Layer 4 — LLM Judge

This is your evidence-grounded judge. The prompt structure matters a lot here:

```
System: You are a fact-verification assistant.
You will be given a question, an AI-generated answer,
and retrieved evidence from trusted sources.
Your job is to assess whether the answer is factually accurate
based solely on the evidence provided.

User:
QUESTION: {question}
ANSWER: {answer}
EVIDENCE: {evidence}

Respond in JSON:
{
  "score": 0–100,         // 0 = definitely hallucination, 100 = fully verified
  "verdict": "verified" | "likely_hallucination" | "unverifiable",
  "explanation": "...",   // 1–2 sentences, evidence-grounded
  "flag": true | false    // flag if score < 60
}
```

Layer 5 — Response Builder

Formats the judge's raw JSON into your final API response. Applies thresholds:

Score	Verdict
75–100	✅ Likely accurate
40–74	⚠️ Uncertain, verify
0–39	🚩 High hallucination risk

Also appends a `source_used` field (Wikipedia/SerpAPI/none) so the frontend can show provenance.

Build Order

1. **Set up FastAPI skeleton** — `/verify` endpoint, Pydantic models, `.env` for API keys
2. **Wire in the LLM Judge** directly first (skip retrieval) — this gives you a working baseline fast
3. **Add Wikipedia retrieval** — test it improves accuracy over the baseline
4. **Add SerpAPI** with the routing logic
5. **Add the Query Preprocessor** claim-extraction step
6. **Add the Evidence Aggregator** (dedup + trim)
7. **Add caching** (Redis or simple in-memory) to avoid repeat API calls for the same question

Key Design Decisions to Make

The biggest one is **how to handle "unverifiable" claims** — when neither Wikipedia nor SerpAPI finds relevant evidence. You have two options: return a neutral score (50) with "no source found," or treat absence of evidence as a weak signal of hallucination risk. I'd recommend the former — it's more honest.

The second is **whether the judge prompt asks for a single score or per-claim scores**. Per-claim is more precise but harder to aggregate into a single UI element. Starting with a single score is fine.